

Reading TaylorScan.v

Zimmermann's hard-to-round search in Rocq, for a beginner

The file is `code/APaul/rocq/TaylorScan.v`. It states the search of `doc/htr.md` and proves that the search part of it misses nothing. This note explains the idea, then the file line by line. The full file is in the annex.

1 The problem

A double x is **hard to round** when $\exp x$ is extremely close to a point where rounding changes its mind. Measured in a unit v (half the distance between two doubles near $\exp x$), the rounding points are the integers, so x is hard to round at level m when $\exp(x)/v$ is within 2^{-m} of an integer. We want every such x in a given range, without evaluating $\exp$ at each of the billions of doubles.

2 Zimmermann's idea in four steps

1. **Cut the range** into subranges of n consecutive doubles $x_0, x_0 + u, \dots, x_0 + (n-1)u$, where u is the distance between two doubles. We number them $j = 0, \dots, n-1$.
2. **Replace exp by a polynomial in j** . On a subrange, $\exp(x_0 + ju)/v$ is very close to a polynomial of small degree in j (its Taylor polynomial). Multiply by a large power of 2, $M = 2^{64\ell}$, and round the coefficients: we get a polynomial with **integer** coefficients,

$$P(j) = A_0 + A_1j + \dots + A_{k-1}j^{k-1}.$$

Only the part of $P(j)$ modulo M matters, because only the distance to an integer matters.

3. **Test each j** . $x_0 + ju$ is a **candidate** when $P(j)$ is within a margin E of a multiple of M . E is chosen to cover everything we neglected: the target 2^{-m} , the Taylor remainder, and the rounding of the coefficients.
4. **Compute $P(0), P(1), P(2), \dots$ with additions only**, by the table of differences (next section). This is what makes the search fast.

The test in one formula. “ b is within E of a multiple of M ” is checked as

$$(b + E) \bmod M \leq 2E.$$

Example with $M = 100$ and $E = 5$: $b = 197$ is within 3 of 200, and $(197 + 5) \bmod 100 = 2 \leq 10$: a candidate. $b = 150$ gives $(150 + 5) \bmod 100 = 55 > 10$: not a candidate. Adding E shifts the interval $[-E, E]$ around a multiple of M to $[0, 2E]$, which a single comparison can test.

3 The table of differences

The **difference** of a sequence f is $\Delta f(j) = f(j+1) - f(j)$. For a polynomial of degree d , taking d differences leaves a constant, and $d+1$ differences leave 0. The **table** at j is the list $f(j), \Delta f(j), \Delta^2 f(j), \dots$

Example with $f(j) = j^2$: the table at $j = 0$ is $[0, 1, 2]$ (the value 0, then $1 - 0 = 1$, then $(4 - 1) - (1 - 0) = 2$). **One step** adds each row to the one below it:

j	$f(j)$	$\Delta f(j)$	$\Delta^2 f(j)$
0	0	1	2
1	$0 + 1 = 1$	$1 + 2 = 3$	2
2	$1 + 3 = 4$	$3 + 2 = 5$	2
3	$4 + 5 = 9$	$5 + 2 = 7$	2

The first column gives 0, 1, 4, 9: the squares, with additions only. This is `Shift.v`, already proved:

- `dtab d f j` is the table of f at j , with $d + 1$ rows;
- `tstep` is one step;
- `tstepE`: if f has degree at most d (`degle d f`), one step turns the table at j into the table at $j + 1$.

4 The file, line by line

Notations used. `int` is the type of integers of the library `mathcomp`. `x *+ n` is x added n times, that is nx . `(a %% M)%Z` is $a \bmod M$, between 0 and $M - 1$. ``|x|` is the absolute value. `iota 0 n` is the list  $0, 1, \dots, n - 1$ , and `[seq j <- s | p j]` keeps the elements  $j$  of  $s$  for which  $p j$  holds. `j \in s` means  $j$  is in the list  $s$ . `(j < n)%N` is a comparison of natural numbers.

The data. One subrange is described by four values:

Variables `(A : seq int) (M E : int) (n : nat)`.

the coefficients $A_0, \dots, A_{k-1}$ as a list, the modulus M , the margin E , and the number n of doubles.

The polynomial.

Definition `P : nat -> int := hpoly A`.

`hpoly` (from `Shift.v`) is the polynomial with coefficients `A` in Horner form: $A_0 + j(A_1 + j(A_2 + \dots))$. Its degree is at most the length of `A`:

Definition `dg := size A`.

Lemma `degleP : degle dg P`.

The test, the formula of section 2:

Definition `hit (b : int) : bool := ((b + E) %% M)%Z <= E *+ 2`.

The walk, step 6 of the note. It keeps the current table `t` and the current index `j`, tests the first row, then takes one step; `c` counts the doubles left:

```
Fixpoint walk (j : nat) (t : seq int) (c : nat) : seq nat :=
  if c is c'.+1 then
    let rest := walk j.+1 (tstep t) c' in
    if hit (nth 0 t 0) then j :: rest else rest
  else [::].
```

`nth 0 t 0` is the first row, the current value $P(j)$. The result is the list of candidates.

The search starts the walk from the table of P at 0:

Definition `scan : seq nat := walk 0 (dtab dg P 0) n`.

5 What is proved

The walk computes the right thing.

Lemma `scanE : scan = [seq j <- iota 0 n | hit (P j)]`.

The walk returns exactly the $j < n$ whose $P(j)$ passes the test: the fast computation with additions gives the same answer as evaluating P at every j . The proof (`walkE`) goes by induction on the number of doubles left; at each step `tstepE` says the table after one step is the table at the next index, whose first row is $P(j + 1)$.

The test catches every b close to a multiple of M .

Lemma `hitP (b w : int) : E *+ 2 < M -> `|b - M * w| <= E -> hit b`.

If $|b - Mw| \leq E$, then $b - Mw + E$ is between 0 and $2E$, which is less than M , so it is $(b + E) \bmod M$. The condition $2E < M$ makes sure the interval $[0, 2E]$ does not wrap around.

The main theorem.

```
Theorem scan_complete j (w : int) :
  E * 2 < M -> (j < n)%M -> `|P j - M * w| <= E -> j \in scan.
```

If $P(j)$ is within E of a multiple of M , then j is a candidate. It is `scanE` and `hitP` put together. The file has no `Admitted`.

6 What is not proved yet

`scan_complete` talks about P , not about `exp`. To reach `exp`, one more statement is needed: **if $x_0 + ju$ is hard to round, then $P(j)$ is within E of a multiple of M** . It holds under the three assumptions of the note:

- (H_A) each A_i is within 1 of M times the fractional part of the exact coefficient $a_i = \exp(x_0)u^i/(i!v)$: this is the evaluation of `exp`, done once per subrange;
- (H_T) the Taylor remainder is at most a known ρ ;
- (H_E) $E \geq M(2^{-m} + \rho) + (1 + n + \dots + n^{k-1})$.

Its proof is arithmetic on real numbers: $Ma_i j^i$ and $M \text{frac}(a_i)j^i$ differ by a multiple of M , because j^i is an integer; each A_i adds an error below $j^i < n^i$; the remainder adds at most $M\rho$. It is written at the end of the file as a comment.

Two details also separate the file from the C program `htr2_fix.c`:

- the C builds the first table in place from $P(0), \dots, P(k-1)$ (steps 3 and 4 of the note), where the file takes `dtab` directly; and the file's table has one more row, which is 0;
- the C reduces every number modulo M after each addition, where the file keeps exact integers and reduces only in the test; the two give the same test, because reduction modulo M commutes with addition.

A Annex: `TaylorScan.v`

```
(** * Zimmermann's search (doc/htr.md): the scan of one subrange
```

```
On a subrange of [n] consecutive doubles [x0 + j u], the note replaces
[exp(x0 + j u) / v] by an integer polynomial in [j],
[P j = A_0 + A_1 j + ... + A_(k-1) j^(k-1)],
whose coefficients are the [A_i] of the note, and walks it with the
table of differences of [Shift.v]. A [j] is a candidate when
[(P j + E) mod M <= 2 E], with [M = beta^l].
```

```
This file states the algorithm over the integers and proves the
part that does not depend on [exp]:
- [scanE]: the walk returns exactly the [j < n] passing the test;
- [scan_complete]: every [j] with [P j] within [E] of a multiple of
[M] is returned.
```

```
What is left to connect it to [exp] is stated at the end. *)
```

```
From mathcomp Require Import all_ssreflect all_algebra.
From APaulRocq Require Import Shift.
```

```
Set Implicit Arguments.
Unset Strict Implicit.
Unset Printing Implicit Defensive.
```

```

Import GRing.Theory Num.Theory.
Local Open Scope ring_scope.

Section Scan.

(** The data of a subrange: the [A_i], least significant first, the
    modulus [M = beta^l], the window [E], and the number of doubles. *)
Variables (A : seq int) (M E : int) (n : nat).

(** The polynomial, in Horner form. *)
Definition P : nat -> int := hpoly A.

(** Its degree is at most [size A], so a table of [size A + 1] rows walks
    it exactly (the note uses [size A] rows: the last one here is [0]). *)
Definition dg := size A.

Lemma degleP : degle dg P.
Proof. exact: degle_hpoly. Qed.

(** The test on the first row of the table: steps 5 and 6 of the note. *)
Definition hit (b : int) : bool := ((b + E) %% M)%Z <= E *+ 2.

(** The main loop, step 6: test the first row, then shift the table. *)
Fixpoint walk (j : nat) (t : seq int) (c : nat) : seq nat :=
  if c is c'.+1 then
    let rest := walk j.+1 (tstep t) c' in
    if hit (nth 0 t 0) then j :: rest else rest
  else [::].

(** The search: the table at [0], steps 3 and 4, then the walk. *)
Definition scan : seq nat := walk 0 (dtab dg P 0) n.

(** The walk returns exactly the [j] in [[i, i + c]] whose [P j] passes
    the test. *)
Lemma walkE i c : walk i (dtab dg P i) c = [seq j <- iota i c | hit (P j)].
Proof.
elim: c i => [//|c IH] i /=.
rewrite tstepE; last exact: degleP.
by rewrite IH.
Qed.

Lemma scanE : scan = [seq j <- iota 0 n | hit (P j)].
Proof. exact: walkE. Qed.

(** If [b] is within [E] of a multiple [M w] of [M], and [2 E < M], then
    [b] passes the test: [(b + E) mod M] is [b - M w + E], in [[0, 2 E]]. *)
Lemma hitP (b w : int) : E *+ 2 < M -> `|b - M * w| <= E -> hit b.
Proof.
move=> EM; rewrite ler_norml => /andP[l r].
have lo : 0 <= b - M * w + E by rewrite -lerB1Dr sub0r.
have hi : b - M * w + E < M.
  by apply: Order.POrderTheory.le_lt_trans EM; rewrite mulr2n lerD2r.
rewrite /hit.
have -> : b + E = w * M + (b - M * w + E).
  by rewrite [w * M]mulrC addrA [M * w + _]addrC subrK.

```

```

rewrite modzMDl modz_small ?lo ?hi //.
by rewrite mulr2n lerD2r.
Qed.

(** The completeness of the scan, with no reference to [exp]. *)
Theorem scan_complete j (w : int) :
  E *+ 2 < M -> (j < n)%N -> `|P j - M * w| <= E -> j \in scan.
Proof.
move=> EM jn Pw.
by rewrite scanE mem_filter (hitP EM Pw) /= mem_iota /= add0n.
Qed.

End Scan.

(** ** What is left: from [exp] to [scan_complete]'s hypothesis

Write [a_i = exp(x0) u^i / (i! v)], [rho] a bound on the Taylor
remainder, and [frac] the fractional part. The note's hypotheses are
(H_A) [|A_i - M frac(a_i)| < 1] the evaluation of exp,
(H_T) [|exp(x0 + j u)/v - sum_i a_i j^i| <= rho] for [j <= n],
(H_E) [E >= M (2^-m + rho) + (1 + n + ... + n^(k-1))].
The lemma to prove, over the reals:
if [dist(exp(x0 + j u)/v, Z) < 2^-m] and [j < n], then there is an
integer [w] with [|P j - M w| <= E].
Its proof: [M a_i j^i] and [M frac(a_i) j^i] differ by a multiple of
[M], since [j^i] is an integer; [|M frac(a_i) - A_i| j^i < n^i]; add
(H_T) times [M]; take [w] from the nearest integer to [exp/v]. *)

```